Valuation model(Made up of many model)

```
package models

import (
"time"

"github.com/google/uuid"
"gorm.io/gorm"
)

// Enum types for better type safety
type PartnerType string
type AgentType string
type StatusType string

const (
// Partner types
PartnerTypeInsurance PartnerType = "insurance"
PartnerTypeBank PartnerType = "bank"
PartnerTypeDealer PartnerType = "dealer"
PartnerTypeOther PartnerType = "other"

// Agent types
AgentTypeIndividual AgentType = "individual"
AgentTypeCorporate AgentType = "corporate"

// Status types
StatusActive StatusType = "active"
StatusInactive StatusType = "inactive"
StatusPending StatusType = "pending"
StatusApproved StatusType = "approved"
StatusRejected StatusType = "rejected"
StatusPaid StatusType = "paid"
StatusUnpaid StatusType = "unpaid"
)

// Customer represents customer data for valuation service
type Customer struct {
ID string `json:"id" gorm:"type:uuid;primary_key"`
Email string `json:"email" gorm:"not null;unique"`
Phone string `json:"phone" gorm:"not null;unique"`
FirstName string `json:"first_name" gorm:"not null"`
LastName string `json:"last_name" gorm:"not null"`
Status string `json:"status" gorm:"not null"`
```

```
CreatedAt time.Time `json:"created_at"`
UpdatedAt time.Time `json:"updated_at"`
DeletedAt *gorm.DeletedAt `json:"deleted_at,omitempty" gorm:"index"`

// Relationships
ValuationRequests []ValuationRequests `json:"valuation_requests"
gorm:"foreignKey:CustomerID"`
VehicleInsurances []VehicleInsurances `json:"vehicle_insurances"
gorm:"foreignKey:CustomerID"`
}

// Partners represents partner organizations
type Partners struct {
ID string `json:"id" gorm:"type:uuid;primary_key;default:gen_random_uuid()"`
Name string `json:"name" gorm:"not null"`
PartnerType PartnerType `json:"partner_type" gorm:"not null"`
ContactPersonName string `json:"contact_person_name" gorm:"not null"`
ContactPersonPhone string `json:"contact_person_phone" gorm:"not null"`
CompanyPhone string `json:"company_phone" gorm:"not null"`
CompanyEmail string `json:"company_email" gorm:"not null"`
LocationAddress string `json:"location_address" gorm:"not null"`
Status StatusType `json:"status" gorm:"default:active"`
CreatedAt time.Time `json:"created_at"`
UpdatedAt time.Time `json:"updated_at"`

// Relationships
Agents []Agents `json:"agents" gorm:"foreignKey:PartnerID"`
ValuationRequests []ValuationRequests `json:"valuation_requests"
gorm:"foreignKey:PartnerID"`
}

// Agents represents partner agents
type Agents struct {
ID string `json:"id" gorm:"type:uuid;primary_key;default:gen_random_uuid()"`
Name string `json:"name" gorm:"not null"`
AgentType AgentType `json:"agent_type" gorm:"not null"`
PartnerID string `json:"partner_id" gorm:"not null"`
ContactPersonName string `json:"contact_person_name" gorm:"not null"`
ContactPersonPhone string `json:"contact_person_phone" gorm:"not null"`
CompanyPhone string `json:"company_phone" gorm:"not null"`
CompanyEmail string `json:"company_email" gorm:"not null"`
LocationAddress string `json:"location_address" gorm:"not null"`
Status StatusType `json:"status" gorm:"default:active"`
CreatedAt time.Time `json:"created_at"`
```

```
UpdatedAt time.Time `json:"updated_at"`

// Relationships
Partner Partners `json:"partner" gorm:"foreignKey:PartnerID"`
ValuationRequests []ValuationRequests `json:"valuation_requests"
gorm:"foreignKey:AgentID"`
}

// ValuationTypes represents different types of valuations
type ValuationTypes struct {
ID string `json:"id" gorm:"type:uuid;primary_key;default:gen_random_uuid()"`
Name string `json:"name" gorm:"not null;unique"`
Status StatusType `json:"status" gorm:"default:active"`
CreatedAt time.Time `json:"created_at"`
UpdatedAt time.Time `json:"updated_at"`
}

// Branches represents company branches
type Branches struct {
ID string `json:"id" gorm:"type:uuid;primary_key;default:gen_random_uuid()"`
Name string `json:"name" gorm:"not null"`
Location string `json:"location" gorm:"not null"`
Status StatusType `json:"status" gorm:"default:active"`
CreatedAt time.Time `json:"created_at"`
UpdatedAt time.Time `json:"updated_at"`
}

// VehicleInsurances represents vehicle insurance information
type VehicleInsurances struct {
ID string `json:"id" gorm:"type:uuid;primary_key;default:gen_random_uuid()"`
InsuranceCompany string `json:"insurance_company" gorm:"not null"`
PolicyNumber string `json:"policy_number" gorm:"unique;not null"`
VehicleID string `json:"vehicle_id" gorm:"not null"`
CustomerID string `json:"customer_id" gorm:"not null"`
ExpiryDate time.Time `json:"expiry_date" gorm:"not null"`
Status StatusType `json:"status" gorm:"default:active"`
CreatedAt time.Time `json:"created_at"`
UpdatedAt time.Time `json:"updated_at"`

// Relationships
Customer Customer `json:"customer" gorm:"foreignKey:CustomerID"`
}

// Valuer represents professional valuers
```

```
type Valuer struct {
ID string `json:"id" gorm:"type:uuid;primary_key;default:gen_random_uuid()"`
Name string `json:"name" gorm:"not null"`
Phone string `json:"phone" gorm:"not null"`
Location string `json:"location" gorm:"not null"`
Status StatusType `json:"status" gorm:"default:active"`
CreatedAt time.Time `json:"created_at"`
UpdatedAt time.Time `json:"updated_at"`

// Relationships
ValuationRequests []ValuationRequests `json:"valuation_requests"
gorm:"foreignKey:ValuerID"`
}

// ValuationRequests represents valuation requests
type ValuationRequests struct {
ID string `json:"id" gorm:"type:uuid;primary_key;default:gen_random_uuid()"`
VehicleID string `json:"vehicle_id" gorm:"not null"`
PartnerID *string `json:"partner_id,omitempty"`
AgentID *string `json:"agent_id,omitempty"`
CustomerID string `json:"customer_id" gorm:"not null"`
LocationAddress string `json:"location_address" gorm:"not null"`
LocationCoordinates *string `json:"location_coordinates,omitempty"`
ContactPersonName string `json:"contact_person_name" gorm:"not null"`
ContactPersonPhone string `json:"contact_person_phone" gorm:"not null"`
ValuerID *string `json:"valuer_id,omitempty"`
ApproverID *string `json:"approver_id,omitempty"`
Status StatusType `json:"status" gorm:"default:pending"`
CreatedAt time.Time `json:"created_at"`
UpdatedAt time.Time `json:"updated_at"`

// Relationships
Partner *Partners `json:"partner,omitempty" gorm:"foreignKey:PartnerID"`
Agent *Agents `json:"agent,omitempty" gorm:"foreignKey:AgentID"`
Customer Customer `json:"customer" gorm:"foreignKey:CustomerID"`
Valuer *Valuer `json:"valuer,omitempty" gorm:"foreignKey:ValuerID"`
ValuationInvoices []ValuationInvoices `json:"valuation_invoices"
gorm:"foreignKey:ValuationRequestID"`
}

// ValuationInvoices represents invoices for valuation services
type ValuationInvoices struct {
ID string `json:"id" gorm:"type:uuid;primary_key;default:gen_random_uuid()"`
ValuationRequestID string `json:"valuation_request_id" gorm:"not null"`
```

```
InvoiceNumber string `json:"invoice_number" gorm:"unique;not null"`
AmountBilled float64 `json:"amount_billed" gorm:"not null"`
Status StatusType `json:"status" gorm:"default:unpaid"`
CreatedAt time.Time `json:"created_at"`
UpdatedAt time.Time `json:"updated_at"`

// Relationships
ValuationRequest ValuationRequests `json:"valuation_request"
gorm:"foreignKey:ValuationRequestID"`
//ValuationPayments []ValuationPayments `json:"valuation_payments"
gorm:"foreignKey:InvoiceNumber;references:InvoiceNumber"`
}

// ValuationPayments represents payments for valuation invoices
type ValuationPayments struct {
ID string `json:"id" gorm:"type:uuid;primary_key;default:gen_random_uuid()"`
InvoiceNumber string `json:"invoice_number" gorm:"not null"`
ReceiptNumber string `json:"receipt_number" gorm:"unique;not null"`
AmountPaid float64 `json:"amount_paid" gorm:"not null"`
Status StatusType `json:"status" gorm:"default:paid"`
CreatedAt time.Time `json:"created_at"`
UpdatedAt time.Time `json:"updated_at"`

// Relationships - Note: Reference by InvoiceNumber instead of ID
//ValuationInvoice ValuationInvoices `json:"valuation_invoice"
gorm:"foreignKey:InvoiceNumber;references:InvoiceNumber"`
}

// BeforeCreate hooks for UUID generation
func (c *Customer) BeforeCreate(tx *gorm.DB) error {
if c.ID == "" {
c.ID = uuid.New().String()
}
return nil
}

func (p *Partners) BeforeCreate(tx *gorm.DB) error {
if p.ID == "" {
p.ID = uuid.New().String()
}
return nil
}

func (a *Agents) BeforeCreate(tx *gorm.DB) error {
```

```
if a.ID == "" {
a.ID = uuid.New().String()
}
return nil
}

func (vt *ValuationTypes) BeforeCreate(tx *gorm.DB) error {
if vt.ID == "" {
vt.ID = uuid.New().String()
}
return nil
}

func (b *Branches) BeforeCreate(tx *gorm.DB) error {
if b.ID == "" {
b.ID = uuid.New().String()
}
return nil
}

func (vi *VehicleInsurances) BeforeCreate(tx *gorm.DB) error {
if vi.ID == "" {
vi.ID = uuid.New().String()
}
return nil
}

func (v *Valuer) BeforeCreate(tx *gorm.DB) error {
if v.ID == "" {
v.ID = uuid.New().String()
}
return nil
}

func (vr *ValuationRequests) BeforeCreate(tx *gorm.DB) error {
if vr.ID == "" {
vr.ID = uuid.New().String()
}
return nil
}

func (vi *ValuationInvoices) BeforeCreate(tx *gorm.DB) error {
if vi.ID == "" {
vi.ID = uuid.New().String()
```

```
}
return nil
}

func (vp *ValuationPayments) BeforeCreate(tx *gorm.DB) error {
if vp.ID == "" {
vp.ID = uuid.New().String()
}
return nil
}
```

Customer model CRUD operations codes.
1. customer_handler.go

```
package handlers

import (
"fmt"

"github.com/Automobile-Association-of-Kenya/aa-core-system/internal/valuation-
service/services"
"github.com/Automobile-Association-of-Kenya/aa-core-system/pkg/response"
"github.com/gin-gonic/gin"
"github.com/go-playground/validator/v10"
)

type CustomerHandler struct {
customerService *services.CustomerService
validator *validator.Validate
}

func NewCustomerHandler(customerService *services.CustomerService)
*CustomerHandler {
return &CustomerHandler{
customerService: customerService,
validator: validator.New(),
}
}

// CreateCustomer creates a new customer
func (h *CustomerHandler) CreateCustomer(c *gin.Context) {
var req services.CreateCustomerRequest
if err := c.ShouldBindJSON(&req); err != nil {
response.BadRequest(c, "Invalid request body")
```

```
return
}

if err := h.validator.Struct(req); err != nil {
response.ValidationError(c, err.Error())
return
}

customer, err := h.customerService.CreateCustomer(req)
if err != nil {
response.BadRequest(c, err.Error())
return
}

response.Success(c, "Customer created successfully", customer)
}

// GetAllCustomers retrieves all customers with pagination
func (h *CustomerHandler) GetAllCustomers(c *gin.Context) {
// Parse query parameters for pagination
limit := 10
offset := 0
if l := c.Query("limit"); l != "" {
if parsedLimit, err := parseInt(l); err == nil && parsedLimit > 0 {
limit = parsedLimit
}
}
if o := c.Query("offset"); o != "" {
if parsedOffset, err := parseInt(o); err == nil && parsedOffset >= 0 {
offset = parsedOffset
}
}

customers, total, err := h.customerService.GetAllCustomers(limit, offset)
if err != nil {
response.InternalServerError(c, "Failed to retrieve customers")
return
}

data := map[string]interface{}{
"customers": customers,
"total": total,
"limit": limit,
"offset": offset,
```

```
}

response.Success(c, "Customers retrieved successfully", data)
}

// GetCustomerByID retrieves a customer by ID
func (h *CustomerHandler) GetCustomerByID(c *gin.Context) {
customerID := c.Param("id")

customer, err := h.customerService.GetCustomerByID(customerID)
if err != nil {
response.NotFound(c, "Customer not found")
return
}

response.Success(c, "Customer retrieved successfully", customer)
}

// UpdateCustomer updates an existing customer
func (h *CustomerHandler) UpdateCustomer(c *gin.Context) {
customerID := c.Param("id")

var req services.UpdateCustomerRequest
if err := c.ShouldBindJSON(&req); err != nil {
response.BadRequest(c, "Invalid request body")
return
}

if err := h.validator.Struct(req); err != nil {
response.ValidationError(c, err.Error())
return
}

customer, err := h.customerService.UpdateCustomer(customerID, req)
if err != nil {
response.BadRequest(c, err.Error())
return
}

response.Success(c, "Customer updated successfully", customer)
}

// DeleteCustomer soft deletes a customer
func (h *CustomerHandler) DeleteCustomer(c *gin.Context) {
```

```
customerID := c.Param("id")

err := h.customerService.DeleteCustomer(customerID)
if err != nil {
response.BadRequest(c, err.Error())
return
}

response.Success(c, "Customer deleted successfully", nil)
}

// SearchCustomers searches customers by query
func (h *CustomerHandler) SearchCustomers(c *gin.Context) {
query := c.Query("q")
if query == "" {
response.BadRequest(c, "Search query is required")
return
}

// Parse pagination parameters
limit := 10
offset := 0
if l := c.Query("limit"); l != "" {
if parsedLimit, err := parseInt(l); err == nil && parsedLimit > 0 {
limit = parsedLimit
}
}
if o := c.Query("offset"); o != "" {
if parsedOffset, err := parseInt(o); err == nil && parsedOffset >= 0 {
offset = parsedOffset
}
}

customers, err := h.customerService.SearchCustomers(query, limit, offset)
if err != nil {
response.InternalServerError(c, "Failed to search customers")
return
}

data := map[string]interface{}{
"customers": customers,
"query": query,
"limit": limit,
"offset": offset,
```

```
}

response.Success(c, "Customer search completed successfully", data)
}

// GetCustomersByStatus retrieves customers by status
func (h *CustomerHandler) GetCustomersByStatus(c *gin.Context) {
status := c.Param("status")

// Parse pagination parameters
limit := 10
offset := 0
if l := c.Query("limit"); l != "" {
if parsedLimit, err := parseInt(l); err == nil && parsedLimit > 0 {
limit = parsedLimit
}
}
if o := c.Query("offset"); o != "" {
if parsedOffset, err := parseInt(o); err == nil && parsedOffset >= 0 {
offset = parsedOffset
}
}

customers, err := h.customerService.GetCustomersByStatus(status, limit, offset)
if err != nil {
response.InternalServerError(c, "Failed to retrieve customers by status")
return
}

data := map[string]interface{}{
"customers": customers,
"status": status,
"limit": limit,
"offset": offset,
}

response.Success(c, "Customers by status retrieved successfully", data)
}

// Helper function to parse integer from string
func parseInt(s string) (int, error) {
var result int
_, err := fmt.Sscanf(s, "%d", &result)
return result, err
```

```
}
```

2. customer_service.go

```
package services

import (
"fmt"
"time"

"github.com/Automobile-Association-of-Kenya/aa-core-system/internal/shared/kafka"
"github.com/Automobile-Association-of-Kenya/aa-core-system/internal/valuation-service/models"
"github.com/Automobile-Association-of-Kenya/aa-core-system/internal/valuation-service/repositories"
"github.com/google/uuid"
)

type CustomerService struct {
customerRepo *repositories.CustomerRepository
kafkaProducer *kafka.Producer
}

func NewCustomerService(customerRepo *repositories.CustomerRepository, kafkaProducer *kafka.Producer) *CustomerService {
return &CustomerService{
customerRepo: customerRepo,
kafkaProducer: kafkaProducer,
}
}

// Request/Response structs
type CreateCustomerRequest struct {
Email string `json:"email" validate:"required,email"`
Phone string `json:"phone" validate:"required"`
FirstName string `json:"first_name" validate:"required"`
LastName string `json:"last_name" validate:"required"`
Status string `json:"status,omitempty"`
}

type UpdateCustomerRequest struct {
Email *string `json:"email,omitempty" validate:"omitempty,email"`
Phone *string `json:"phone,omitempty"`
FirstName *string `json:"first_name,omitempty"`
LastName *string `json:"last_name,omitempty"`
```

```
Status *string `json:"status,omitempty"`
}

// CreateCustomer creates a new customer
func (s *CustomerService) CreateCustomer(req CreateCustomerRequest)
(*models.Customer, error) {
// Check if customer with email already exists
existing, err := s.customerRepo.GetCustomerByEmail(req.Email)
if err == nil && existing != nil {
return nil, fmt.Errorf("customer with email %s already exists", req.Email)
}

// Check if customer with phone already exists
existing, err = s.customerRepo.GetCustomerByPhone(req.Phone)
if err == nil && existing != nil {
return nil, fmt.Errorf("customer with phone %s already exists", req.Phone)
}

// Set default status if not provided
status := req.Status
if status == "" {
status = "active"
}

// Create customer
customer := &models.Customer{
Email: req.Email,
Phone: req.Phone,
FirstName: req.FirstName,
LastName: req.LastName,
Status: status,
}

if err := s.customerRepo.CreateCustomer(customer); err != nil {
return nil, fmt.Errorf("failed to create customer: %v", err)
}

// Publish customer created event
s.publishCustomerEvent("customer.created", customer)

return customer, nil
}

// GetAllCustomers retrieves all customers with pagination
```

```
func (s *CustomerService) GetAllCustomers(limit, offset int) ([]models.Customer,
int64, error) {
customers, err := s.customerRepo.GetAllCustomers(limit, offset)
if err != nil {
return nil, 0, fmt.Errorf("failed to retrieve customers: %v", err)
}

total, err := s.customerRepo.GetCustomersCount()
if err != nil {
return nil, 0, fmt.Errorf("failed to get customers count: %v", err)
}

return customers, total, nil
}

// GetCustomerByID retrieves a customer by ID
func (s *CustomerService) GetCustomerByID(customerID string) (*models.Customer,
error) {
customer, err := s.customerRepo.GetCustomerByID(customerID)
if err != nil {
return nil, fmt.Errorf("customer not found: %v", err)
}

return customer, nil
}

// UpdateCustomer updates an existing customer
func (s *CustomerService) UpdateCustomer(customerID string, req
UpdateCustomerRequest) (*models.Customer, error) {
// Get existing customer
customer, err := s.customerRepo.GetCustomerByID(customerID)
if err != nil {
return nil, fmt.Errorf("customer not found: %v", err)
}

// Check for email conflicts if email is being updated
if req.Email != nil && *req.Email != customer.Email {
existing, err := s.customerRepo.GetCustomerByEmail(*req.Email)
if err == nil && existing != nil && existing.ID != customerID {
return nil, fmt.Errorf("customer with email %s already exists", *req.Email)
}
customer.Email = *req.Email
}
```

```
// Check for phone conflicts if phone is being updated
if req.Phone != nil && *req.Phone != customer.Phone {
existing, err := s.customerRepo.GetCustomerByPhone(*req.Phone)
if err == nil && existing != nil && existing.ID != customerID {
return nil, fmt.Errorf("customer with phone %s already exists", *req.Phone)
}
customer.Phone = *req.Phone
}

// Update other fields
if req.FirstName != nil {
customer.FirstName = *req.FirstName
}
if req.LastName != nil {
customer.LastName = *req.LastName
}
if req.Status != nil {
customer.Status = *req.Status
}

// Save updates
if err := s.customerRepo.UpdateCustomer(customer); err != nil {
return nil, fmt.Errorf("failed to update customer: %v", err)
}

// Publish customer updated event
s.publishCustomerEvent("customer.updated", customer)

return customer, nil
}

// DeleteCustomer soft deletes a customer
func (s *CustomerService) DeleteCustomer(customerID string) error {
// Check if customer exists
customer, err := s.customerRepo.GetCustomerByID(customerID)
if err != nil {
return fmt.Errorf("customer not found: %v", err)
}

// Check if customer has any valuation requests
hasRequests, err := s.customerRepo.CustomerHasValuationRequests(customerID)
if err != nil {
return fmt.Errorf("failed to check customer dependencies: %v", err)
}
```

```
if hasRequests {
return fmt.Errorf("cannot delete customer with existing valuation requests")
}

// Soft delete customer
if err := s.customerRepo.DeleteCustomer(customerID); err != nil {
return fmt.Errorf("failed to delete customer: %v", err)
}

// Publish customer deleted event
s.publishCustomerEvent("customer.deleted", customer)

return nil
}

// SearchCustomers searches customers by query
func (s *CustomerService) SearchCustomers(query string, limit, offset int)
([]models.Customer, error) {
customers, err := s.customerRepo.SearchCustomers(query, limit, offset)
if err != nil {
return nil, fmt.Errorf("failed to search customers: %v", err)
}

return customers, nil
}

// GetCustomersByStatus retrieves customers by status
func (s *CustomerService) GetCustomersByStatus(status string, limit, offset int)
([]models.Customer, error) {
customers, err := s.customerRepo.GetCustomersByStatus(status, limit, offset)
if err != nil {
return nil, fmt.Errorf("failed to retrieve customers by status: %v", err)
}

return customers, nil
}

// GetCustomerByEmail retrieves a customer by email
func (s *CustomerService) GetCustomerByEmail(email string) (*models.Customer,
error) {
customer, err := s.customerRepo.GetCustomerByEmail(email)
if err != nil {
return nil, fmt.Errorf("customer not found: %v", err)
```

```
}

return customer, nil
}

// GetCustomerByPhone retrieves a customer by phone
func (s *CustomerService) GetCustomerByPhone(phone string) (*models.Customer,
error) {
customer, err := s.customerRepo.GetCustomerByPhone(phone)
if err != nil {
return nil, fmt.Errorf("customer not found: %v", err)
}

return customer, nil
}

// Event handlers for customer sync from user service
func (s *CustomerService) HandleCustomerCreated(event kafka.Event) error {
customerData := event.Data

customer := &models.Customer{
ID: customerData["user_id"].(string),
Email: customerData["email"].(string),
Phone: customerData["phone"].(string),
FirstName: customerData["first_name"].(string),
LastName: customerData["last_name"].(string),
Status: "pending_verification",
}

if err := s.customerRepo.CreateOrUpdateCustomer(customer); err != nil {
return fmt.Errorf("failed to sync customer from user service: %v", err)
}

return nil
}

func (s *CustomerService) HandleCustomerVerified(event kafka.Event) error {
customerID := event.Data["user_id"].(string)

customer, err := s.customerRepo.GetCustomerByID(customerID)
if err != nil {
return fmt.Errorf("customer not found in valuation service: %v", err)
}
```

```
customer.Status = "active"
return s.customerRepo.UpdateCustomer(customer)
}

// Utility methods
func (s *CustomerService) publishCustomerEvent(eventType string, customer
*models.Customer) {
if s.kafkaProducer == nil {
return
}

event := kafka.Event{
ID: uuid.New().String(),
Type: eventType,
Source: "valuation-service",
Data: map[string]interface{}{
"customer_id": customer.ID,
"email": customer.Email,
"phone": customer.Phone,
"first_name": customer.FirstName,
"last_name": customer.LastName,
"status": customer.Status,
},
Timestamp: time.Now().Unix(),
Version: "1.0",
}

s.kafkaProducer.PublishEvent("customer_events", event)
}
```

3. customer_repository.go

```
package repositories

import (
"github.com/Automobile-Association-of-Kenya/aa-core-system/internal/valuation-
service/models"
"gorm.io/gorm"
)

type CustomerRepository struct {
db *gorm.DB
}

func NewCustomerRepository(db *gorm.DB) *CustomerRepository {
```

```
return &CustomerRepository{db: db}
}

// CreateCustomer creates a new customer
func (r *CustomerRepository) CreateCustomer(customer *models.Customer) error {
return r.db.Create(customer).Error
}

// CreateOrUpdateCustomer creates or updates customer (used for sync from user service)
func (r *CustomerRepository) CreateOrUpdateCustomer(customer *models.Customer) error {
var existingCustomer models.Customer
err := r.db.Where("id = ?", customer.ID).First(&existingCustomer).Error

if err == gorm.ErrRecordNotFound {
return r.db.Create(customer).Error
}

return r.db.Save(customer).Error
}

// GetAllCustomers retrieves all customers with pagination
func (r *CustomerRepository) GetAllCustomers(limit, offset int) ([]models.Customer, error) {
var customers []models.Customer
err := r.db.Where("deleted_at IS NULL").
Order("created_at DESC").
Limit(limit).
Offset(offset).
Find(&customers).Error

return customers, err
}

// GetCustomersCount returns total count of customers
func (r *CustomerRepository) GetCustomersCount() (int64, error) {
var count int64
err := r.db.Model(&models.Customer{}).
Where("deleted_at IS NULL").
Count(&count).Error

return count, err
}
```

```
// GetCustomerByID retrieves a customer by ID
func (r *CustomerRepository) GetCustomerByID(customerID string)
(*models.Customer, error) {
var customer models.Customer
err := r.db.Where("id = ? AND deleted_at IS NULL", customerID).
Preload("ValuationRequests").
Preload("VehicleInsurances").
First(&customer).Error

return &customer, err
}

// GetCustomerByEmail retrieves a customer by email
func (r *CustomerRepository) GetCustomerByEmail(email string) (*models.Customer,
error) {
var customer models.Customer
err := r.db.Where("email = ? AND deleted_at IS NULL", email).First(&customer).Error

return &customer, err
}

// GetCustomerByPhone retrieves a customer by phone
func (r *CustomerRepository) GetCustomerByPhone(phone string)
(*models.Customer, error) {
var customer models.Customer
err := r.db.Where("phone = ? AND deleted_at IS NULL", phone).First(&customer).Error

return &customer, err
}

// UpdateCustomer updates an existing customer
func (r *CustomerRepository) UpdateCustomer(customer *models.Customer) error {
return r.db.Save(customer).Error
}

// DeleteCustomer soft deletes a customer
func (r *CustomerRepository) DeleteCustomer(customerID string) error {
return r.db.Model(&models.Customer{}).
Where("id = ?", customerID).
Update("deleted_at", gorm.DeletedAt{}).Error
}

// SearchCustomers searches customers by email, phone, first name, or last name
```

```
func (r *CustomerRepository) SearchCustomers(query string, limit, offset int)
([]models.Customer, error) {
var customers []models.Customer
searchPattern := "%" + query + "%"

err := r.db.Where("deleted_at IS NULL").
Where("email ILIKE ? OR phone ILIKE ? OR first_name ILIKE ? OR last_name ILIKE ? OR
CONCAT(first_name, ' ', last_name) ILIKE ?",
searchPattern, searchPattern, searchPattern, searchPattern, searchPattern).
Order("created_at DESC").
Limit(limit).
Offset(offset).
Find(&customers).Error

return customers, err
}

// GetCustomersByStatus retrieves customers by status
func (r *CustomerRepository) GetCustomersByStatus(status string, limit, offset int)
([]models.Customer, error) {
var customers []models.Customer
err := r.db.Where("status = ? AND deleted_at IS NULL", status).
Order("created_at DESC").
Limit(limit).
Offset(offset).
Find(&customers).Error

return customers, err
}

// CustomerHasValuationRequests checks if customer has any valuation requests
func (r *CustomerRepository) CustomerHasValuationRequests(customerID string)
(bool, error) {
var count int64
err := r.db.Model(&models.ValuationRequests{}).
Where("customer_id = ?", customerID).
Count(&count).Error

return count > 0, err
}

// GetActiveCustomersCount returns count of active customers
func (r *CustomerRepository) GetActiveCustomersCount() (int64, error) {
var count int64
```

```
err := r.db.Model(&models.Customer{}).
Where("status = ? AND deleted_at IS NULL", "active").
Count(&count).Error

return count, err
}

// GetCustomersCreatedBetween returns customers created between two dates
func (r *CustomerRepository) GetCustomersCreatedBetween(startDate, endDate
string) ([]models.Customer, error) {
var customers []models.Customer
err := r.db.Where("created_at BETWEEN ? AND ? AND deleted_at IS NULL", startDate,
endDate).
Order("created_at DESC").
Find(&customers).Error

return customers, err
}

// BulkUpdateCustomerStatus updates status for multiple customers
func (r *CustomerRepository) BulkUpdateCustomerStatus(customerIDs []string,
status string) error {
return r.db.Model(&models.Customer{}).
Where("id IN ? AND deleted_at IS NULL", customerIDs).
Update("status", status).Error
}

// GetCustomerWithValuationRequests retrieves customer with their valuation
requests
func (r *CustomerRepository) GetCustomerWithValuationRequests(customerID
string) (*models.Customer, error) {
var customer models.Customer
err := r.db.Where("id = ? AND deleted_at IS NULL", customerID).
Preload("ValuationRequests").
Preload("ValuationRequests.Partner").
Preload("ValuationRequests.Agent").
Preload("ValuationRequests.Valuer").
First(&customer).Error

return &customer, err
}

// GetCustomerWithInsurances retrieves customer with their vehicle insurances
```

```
func (r *CustomerRepository) GetCustomerWithInsurances(customerID string)
(*models.Customer, error) {
var customer models.Customer
err := r.db.Where("id = ? AND deleted_at IS NULL", customerID).
Preload("VehicleInsurances").
First(&customer).Error

return &customer, err
}
```

4. routes.go

```
package routes

import (
"github.com/Automobile-Association-of-Kenya/aa-core-system/internal/user-
management/middleware"
"github.com/Automobile-Association-of-Kenya/aa-core-system/internal/valuation-
service/handlers"
"github.com/gin-gonic/gin"
swaggerFiles "github.com/swaggo/files"
ginSwagger "github.com/swaggo/gin-swagger"
)

func SetupRoutes(
r *gin.Engine,
customerHandler *handlers.CustomerHandler,
authMiddleware *middleware.AuthMiddleware,
) {
// Add CORS middleware
r.Use(middleware.CORSMiddleware())

// Add logging middleware
r.Use(middleware.LoggingMiddleware())

// Swagger documentation
r.GET("/docs/*any", ginSwagger.WrapHandler(swaggerFiles.Handler))

// API routes
api := r.Group("/api/v1")

// Public customer reference data (no auth required) - if any
// customers := api.Group("/customers")
// {
```

```
// // Add any public customer endpoints here if needed
// customers.GET("/search", customerHandler.SearchCustomers) // Public search if needed
// }

// Protected routes
protected := api.Group("/")
protected.Use(authMiddleware.RequireAuth())
{
// Customer operations
customers := protected.Group("/customers")
{
// Basic CRUD operations
customers.GET("/customers", customerHandler.GetAllCustomers) // GET /api/v1/admin/customers?limit=10&offset=0
customers.POST("", customerHandler.CreateCustomer) // POST /api/v1/customers
customers.GET("/:id", customerHandler.GetCustomerByID) // GET /api/v1/customers/{id}
customers.PUT("/:id", customerHandler.UpdateCustomer) // PUT /api/v1/customers/{id}
customers.DELETE("/:id", customerHandler.DeleteCustomer) // DELETE /api/v1/customers/{id}
customers.GET("/search", customerHandler.SearchCustomers) // GET /api/v1/customers/search?q=john&limit=10&offset=0
customers.GET("/status/:status", customerHandler.GetCustomersByStatus) // GET /api/v1/customers/status/active?limit=10&offset=0
}

// Admin only routes
admin := protected.Group("/admin")
admin.Use(authMiddleware.RequireRole("admin", "super_admin"))
{
// Admin customer operations
// Add other admin-only customer operations here if needed
// admin.POST("/customers/bulk", customerHandler.BulkCreateCustomers)
// admin.GET("/customers/export", customerHandler.ExportCustomers)
}
}
}
```

5. main.go

```
package main

import (
"context"
"log"
"net/http"
"os"
"os/signal"
"syscall"
"time"

"github.com/Automobile-Association-of-Kenya/aa-core-system/internal/shared/database"
"github.com/Automobile-Association-of-Kenya/aa-core-system/internal/shared/kafka"
"github.com/Automobile-Association-of-Kenya/aa-core-system/internal/shared/redis"
"github.com/Automobile-Association-of-Kenya/aa-core-system/internal/user-management/config"
"github.com/Automobile-Association-of-Kenya/aa-core-system/internal/user-management/middleware"
"github.com/Automobile-Association-of-Kenya/aa-core-system/internal/valuation-service/handlers"
"github.com/Automobile-Association-of-Kenya/aa-core-system/internal/valuation-service/models"
"github.com/Automobile-Association-of-Kenya/aa-core-system/internal/valuation-service/repositories"
"github.com/Automobile-Association-of-Kenya/aa-core-system/internal/valuation-service/routes"
"github.com/Automobile-Association-of-Kenya/aa-core-system/internal/valuation-service/services"
"github.com/Automobile-Association-of-Kenya/aa-core-system/pkg/auth"
"github.com/gin-gonic/gin"
"github.com/joho/godotenv"
"gorm.io/gorm"
)

// @title AA Core System - Valuation Service API
// @version 1.0
// @description Vehicle valuation service for AA Core System
// @contact.name API Support
// @contact.email support@aa.co.ke
// @host localhost:8083
// @BasePath /api/v1
// @securityDefinitions.apikey BearerAuth
```

```
// @in header
// @name Authorization
func main() {
// Load environment variables
if err := godotenv.Load(); err != nil {
log.Println("No .env file found, using system environment variables")
}

// Load configuration
cfg, err := config.LoadConfig("./configs")
if err != nil {
log.Fatalf("Failed to load config: %v", err)
}

// Set Gin mode
gin.SetMode(cfg.Server.Mode)

// Initialize database
dbConfig := database.Config{
Host: cfg.Database.Host,
Port: cfg.Database.Port,
User: cfg.Database.User,
Password: cfg.Database.Password,
DBName: "aa_valuation_service", // Separate database for valuation service
SSLMode: cfg.Database.SSLMode,
MaxOpenConns: cfg.Database.MaxOpenConns,
MaxIdleConns: cfg.Database.MaxIdleConns,
ConnMaxLifetime: cfg.Database.ConnMaxLifetime,
}

db, err := database.NewPostgresConnection(dbConfig)
if err != nil {
log.Fatalf("Failed to connect to database: %v", err)
}

// Run migrations
if err := runMigrations(db); err != nil {
log.Fatalf("Failed to run migrations: %v", err)
}

// Initialize Redis
redisConfig := redis.Config{
Host: cfg.Redis.Host,
Port: cfg.Redis.Port,
```

```
Password: cfg.Redis.Password,
DB: 3, // Different Redis DB for valuation service
PoolSize: cfg.Redis.PoolSize,
}

redisClient, err := redis.NewRedisClient(redisConfig)
if err != nil {
log.Fatalf("Failed to connect to Redis: %v", err)
}

// Initialize Kafka
kafkaConfig := kafka.Config{
Brokers: cfg.Kafka.Brokers,
Topics: map[string]string{
"user_events": "user-events",
"valuation_events": "valuation-events",
},
}

kafkaProducer := kafka.NewProducer(kafkaConfig)
defer kafkaProducer.Close()

// Initialize JWT manager
jwtManager := auth.NewJWTManager(
cfg.JWT.Secret,
cfg.JWT.AccessTokenDuration,
cfg.JWT.RefreshTokenDuration,
)

// Initialize repositories
customerRepo := repositories.NewCustomerRepository(db)
// Add other repositories as you implement them

// Initialize services
customerService := services.NewCustomerService(customerRepo, kafkaProducer)
// Add other services as you implement them

// Initialize middleware
authMiddleware := middleware.NewAuthMiddleware(jwtManager, nil, redisClient)

// Initialize handlers
customerHandler := handlers.NewCustomerHandler(customerService)
// Add other handlers as you implement them
```

```
// Initialize Gin router
router := gin.New()
router.Use(gin.Recovery())

// Health check endpoint (no auth required)
router.GET("/health", func(c *gin.Context) {
c.JSON(200, gin.H{"status": "healthy", "service": "valuation-service"})
})

// Setup routes
routes.SetupRoutes(router, customerHandler, authMiddleware)
// Add other routes as you implement them
// routes.RegisterPartnerRoutes(router, partnerHandler, authMiddleware)
// routes.RegisterAgentRoutes(router, agentHandler, authMiddleware)

// Start Kafka consumers in background
go startKafkaConsumers(kafkaConfig, customerService)

// Create HTTP server
srv := &http.Server{
Addr: ":8083", // Different port for valuation service
Handler: router,
ReadTimeout: cfg.Server.ReadTimeout,
WriteTimeout: cfg.Server.WriteTimeout,
}

// Start server in a goroutine
go func() {
log.Printf("Starting valuation service on port 8083")
if err := srv.ListenAndServe(); err != nil && err != http.ErrServerClosed {
log.Fatalf("Failed to start server: %v", err)
}
}()

// Wait for interrupt signal to gracefully shutdown the server
quit := make(chan os.Signal, 1)
signal.Notify(quit, syscall.SIGINT, syscall.SIGTERM)
<-quit
log.Println("Shutting down valuation service...")

ctx, cancel := context.WithTimeout(context.Background(), 30*time.Second)
defer cancel()

if err := srv.Shutdown(ctx); err != nil {
```

```
log.Fatalf("Server forced to shutdown: %v", err)
}

log.Println("Valuation service shutdown complete")
}

// runMigrations handles database migrations for valuation service
func runMigrations(db *gorm.DB) error {
log.Println("Running valuation service migrations...")

err := db.AutoMigrate(
&models.Customer{},
&models.Partners{},
&models.Agents{},
&models.ValuationTypes{},
&models.Branches{},
&models.VehicleInsurances{},
&models.Valuer{},
&models.ValuationRequests{},
&models.ValuationInvoices{},
&models.ValuationPayments{},
)

if err != nil {
return err
}

log.Println("Valuation service migrations completed successfully")
return nil
}

// startKafkaConsumers handles Kafka event consumption
func startKafkaConsumers(kafkaConfig kafka.Config, customerService
*services.CustomerService) {
consumer := kafka.NewConsumer(kafkaConfig, "valuation-service")

// Handle user events from user-management service
go func() {
err := consumer.ConsumeEvents("user_events", func(event kafka.Event) error {
switch event.Type {
case "user.registered":
return customerService.HandleCustomerCreated(event)
case "user.email_verified":
return customerService.HandleCustomerVerified(event)
```

```
default:
log.Printf("Unhandled event type: %s", event.Type)
}
return nil
})
if err != nil {
log.Printf("Failed to consume user events: %v", err)
}
}()
}
```